Synthetic User Swarm — System Architecture

Oct 27, 2026 · @Anshuman Das

Overview

A Synthetic User Swarm launches 10–100 LLM-driven personas at a live website in parallel. Each persona tries to finish a real task the way that kind of person would. The system turns where they struggle into a ranked list of usability issues, each linked to the screenshots, steps and think-aloud quotes that prove it.

Core rule: no evidence, no issue. The LLM may propose a problem, but it enters the report only if it points at recorded steps that show it. This single rule is what separates a credible tool from an LLM writing plausible UX opinions.

Goals

- Catch task-blocking and confusing UX before real users hit it, cheaply enough to run on every staging deploy.
- Cover the user diversity teams rarely test: mobile, low digital literacy, screen reader, keyboard-only, non-native speakers, impatient users.
- Make every finding reproducible: one click from an issue to the replay at the moment it happened.
- Show the swarm working live, because watching agents struggle is the demo.

Non-goals

- Replacing real user research. The swarm finds "can't do it" and "didn't understand it" problems, not desirability, trust or willingness to pay.
- Functional QA or load testing, although real bugs will surface along the way.
- Testing third-party sites without the owner's permission.

Direction	What
In	Target URL, 1–5 tasks with success criteria, persona mix, budget (sessions, steps, cost cap), optional test accounts

Direction	What
Out	Task × persona success matrix, ranked issues with severity and confidence, evidence per issue, session replays, JSON export

High-level architecture

Four stages run in order: plan the study, run the swarm, analyse the evidence, deliver the outputs. Workers are stateless and parallel, and everything they observe lands in one evidence store that both analysis and the live dashboard read.

Every issue in the report traces back to recorded evidence



system architecture · 4 stages, 12 components

Read it as a snake: plan left to right, run in parallel workers, analyse right to left from the evidence store, then deliver. Because the dashboard and the report read the same store, what the audience sees live is exactly what the report cites.

Persona engine

Personas only behave differently when their traits are enforced in code, not just described in a prompt. "You are 68 and not tech-savvy" alone yields an agent that still reads every word and never misses a button. So every trait maps to a mechanism the runtime enforces: what the agent can see, what it can do, and how long it will try.

Each persona is a card validated against a schema:

```
id: asha-68-mobile
name: Asha
archetype: low-confidence mobile shopper
locale: en-IN, non-native English
device: { viewport: 390x844, touch: true, network: slow-4g }
digital_literacy: 2          # 1-5
domain_knowledge: 3          # 1-5
goal_framing: "Buy a birthday gift under ₹2,000 for my grandson"
reading: skim                # skim | normal | thorough
patience: { max_steps: 25, max_retries_per_obstacle: 2 }
accessibility: { zoom: 150 }
quirks:
  - closes pop-ups without reading them
  - avoids creating an account unless forced
  - does not know words like "SKU" or "cart subtotal"
```

Trait	Enforced through	Example
Device and network	Browser context	390×844 viewport, touch events, throttled network
Vision and zoom	Perception filter	Page zoom 200%; a colour-blind persona sees a daltonised screenshot
Screen reader	Perception filter	No screenshot at all; only the accessibility tree, in reading order
Keyboard-only	Action filter	Tab, Shift+Tab, Enter, Space and arrows only; no pointer actions
Reading style	Perception filter	Skimmers see headings, buttons and the first ~12 words of each text block
Patience	Budget	Hard step cap, retry cap per obstacle, `give_up` always available
Knowledge and language	Prompt	Words it doesn't know; the goal phrased in its own words, not the site's
Quirks	Prompt + policy	Dismisses modals unread; refuses sign-up when a guest path exists

Where personas come from

1. A curated core of 8–12 hand-written archetypes that cover the extremes (screen reader, keyboard-only, first-time mobile user, power user, non-native speaker, impatient user).
2. Generated personas: an LLM fills the schema from a segment description such as "Indian small-business owners, 30–55, mostly Android".
3. Seeded personas: device mix, locales and demographics taken from the site's own analytics or survey data.

The study's mix should mirror real traffic (if 70% of visitors are on mobile, so are 70% of agents) plus deliberate edge cases. Sample across trait axes with a stratified design and drop near-duplicates by embedding similarity, so 50 agents are not 50 copies of the same user.

Orchestrator and swarm scheduler

The orchestrator expands a study into sessions (persona × task × repetition) and runs each as a durable workflow. Twelve personas, two tasks and three repetitions make 72 sessions.

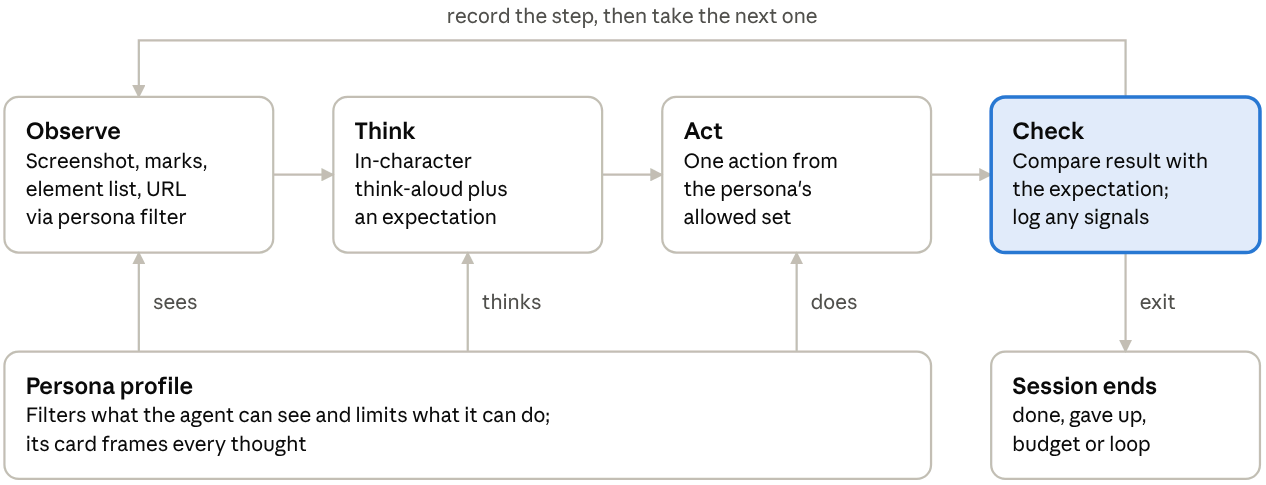
Each session moves through `queued → provisioning → running → succeeded | gave_up | failed | infra_error | timed_out → analysed`.

- **Durable execution.** Each session is a workflow (Temporal, or BullMQ on Redis for an MVP). A crashed worker resumes from the last recorded step instead of restarting the persona.
- **Isolation.** Fresh browser context per session, so no shared cookies, storage or cache. Each worker runs in its own container, and egress goes through a proxy that allows only the study's domains.
- **Politeness.** A per-domain concurrency cap (start at 5–10 parallel sessions), a user agent that identifies the swarm, and backoff on 429 or 5xx responses.
- **Budgets.** Per session: a step cap, a wall-clock cap (about 10 minutes) and a token cap. Per study: a hard cost cap that stops scheduling new sessions once reached.
- **Repetitions.** Run each persona × task three times with different seeds. One failure in three is noise; three in three is a finding.
- **Determinism.** Fixed viewport, timezone, locale and geolocation per persona, and site state reset between sessions (seeded test accounts, emptied carts).
- **Infra failures kept out of findings.** Browser crashes, CAPTCHAs, proxy errors and 5xx storms are tagged `infra_error`, retried, and never counted as usability issues.
- **Auth outside the model.** Logins run as scripted steps with credentials from a secrets vault. The LLM never sees a password; the persona simply starts signed in when the task needs it.

Browser agent runtime

Each agent runs a perceive → think → act → check loop, one browser action per step, until it finishes, gives up or runs out of budget.

Each step ends by checking the result against what the agent expected



agent step loop · 4 stages, persona-shaped

The persona shapes three of the four stages; the check stage is pure code, which is why its signals can be trusted.

What the agent sees each step (after the persona's perception filter)

- A screenshot with Set-of-Marks: a numbered box drawn over every interactive element.
- A compact element list: mark number, role, accessible name, state, and whether it is in view.
- URL, page title, and whether the page changed since the last action.
- The task in the persona's words, the last 5–8 steps, and a short scratchpad of notes.

Sending both a picture and a list is the key choice. The list keeps actions reliable; the picture lets the agent notice what a DOM never shows, like a button hidden behind a cookie banner or text too faint to read.

What the agent returns each step

```
{
  "think_aloud": "I tapped 'Add to bag' but I can't tell if anything
happened.",
  "confidence": 2,
  "frustration": 6,
  "action": { "type": "click", "mark": 14 },
  "expectation": { "says": "The bag count changes to 1", "check": { "type":
"text_changes", "mark": 3 } }
}
```

The `expectation` field is the best friction sensor in the system. It pairs a sentence with a predicate the runtime tests in code after acting: URL changes, text appears, an element changes, a dialog opens. A mismatch is logged as a signal even when the agent later recovers.

Action space

Action	Notes
`click(mark)` / `tap(mark)`	Resolved to a Playwright locator; coordinate fallback if the element moved
`type(mark, text, submit?)`	Typing time is simulated per persona for estimates, never actually slept
`select`, `scroll`, `hover`, `press(key)`	Keyboard-only personas get `press` and nothing pointer-based
`back`	Real users press back; it also counts as a backtracking signal
`wait(seconds)`	For spinners and slow pages
`give_up(reason)`	Always available; the reason becomes evidence
`done(claim)`	The agent's claim of success, checked independently

Leave `goto(url)` out for most personas. Real users don't guess URLs, and letting the agent do it hides navigation problems.

Stopping and success

- A session ends on `done`, `give_up`, the step or time budget, or loop detection (the same page-state hash three times).
- Success is never taken on the agent's word. An independent checker tests the task's criteria: a URL pattern, text on the page ("Order confirmed"), or a network call such as `POST /api/orders` returning 201. An LLM judge on the final screenshot covers fuzzy goals.

Model routing and timing

- A fast, cheap vision model (Haiku-class) takes routine steps. When the agent is stuck, the runtime escalates to a stronger model for two or three steps, then drops back. UXAgent uses a similar split between a fast action loop and a slow reflection loop.
- Time on task is estimated, not measured: reading time from words seen at the persona's reading speed, plus per-action time in the style of the Keystroke-Level Model. Raw LLM latency would distort every timing.

Evidence capture layer

Every step writes an immutable, content-addressed record before the next action runs, so any claim in a report traces back to what was actually on screen.

Artifact	Captured	Format	Why it matters
Screenshots before and after	Every step	WebP, clean and marked versions	The visual proof on every issue card
Agent output	Every step	JSON: think-aloud, expectation, confidence, frustration, action	Quotes, and the expectation-mismatch signal
Target element	Every action	Selector, text, bounding box	Highlights the exact element on the screenshot
Accessibility tree	Every step	JSON snapshot	Screen-reader evidence, missing labels
DOM recording	Continuous	rrweb event stream	Faithful replay of any moment
Console and JS errors	Continuous	Log lines tagged with step id	Ties broken UX to broken code

Artifact	Captured	Format	Why it matters
Network	Continuous	HAR, sliced per step	Failed requests, slow APIs, success checks
Performance	Per page	LCP, layout-shift events	Explains misclicks caused by moving content
Accessibility audit	Each new page	axe-core results	Objective WCAG violations
Full trace	Per session	Playwright trace zip	Developer-grade debugging of any step

- Blobs go to object storage under `study/session/step/` keyed by SHA-256; metadata rows go to Postgres.
- Every step also emits an event on a stream (Redis Streams or NATS) that feeds the live dashboard.
- Issues reference step ids, never copies of evidence, so the evidence can't drift from what happened.
- Redaction runs at capture time: password and card fields masked in screenshots and in rrweb (`maskAllInputs`), emails and phone numbers scrubbed from logs.

Friction detection and issue synthesis

Analysis runs in three passes: cheap deterministic signals, an LLM critic per session, then clustering across the swarm. Objective signals anchor the LLM's interpretation, and clustering turns 72 noisy sessions into a short list of real issues.

Pass 1: deterministic signals (per step, no LLM)

Signal	Starting rule (tune per site)
Dead click	Click with no DOM change, navigation or network call within 1 s
Rage click	3+ clicks on the same element or region within 2 s
Expectation mismatch	The agent's `expectation` predicate fails after the action

Signal	Starting rule (tune per site)
Backtrack	`back`, or a return to an earlier page-state hash
Loop	Same page-state hash 3 times
Error shown	Validation message, 4xx/5xx response, or console exception
Search miss	Site search returns zero results
Scroll hunt	4+ scrolls on one page without acting
Stall	5+ steps with no progress toward the goal
Give-up or failure	`give_up` called, or the success checker fails
Accessibility	axe-core violation, keyboard trap, control with no accessible name
Layout	Primary action off-screen or covered by an overlay at the persona's viewport

Pass 2: LLM critic (per session)

A stronger model reads the session's think-aloud, signals and key screenshots, then proposes candidate issues. Each must cite the step ids it rests on, quote the agent, name the page and element, and tag the heuristic violated (Nielsen's 10). A validator rejects any candidate whose step ids don't exist, whose element isn't in those steps, or that has neither a signal nor a quote behind it.

Pass 3: cross-session synthesis

1. Embed each candidate on its title, description, page path and element selector.
2. Cluster (HDBSCAN or agglomerative), then let an LLM merge pass join clusters that share a root cause.
3. Each cluster becomes one issue with its reach: how many sessions, personas and repetitions hit it.
4. Pick the 3–5 strongest evidence steps for the issue card: the clearest screenshot, the most telling quote, the signal that fired.

Severity and confidence

Severity combines how bad an issue is with how many agents hit it:

$$\text{severity} = \text{impact} \times \left(0.5 + 0.5 \cdot \frac{\text{sessions affected}}{\text{sessions that reached the page}}\right)$$

Impact is 4 when the task was blocked, 3 for a major detour, 2 for minor friction and 1 for cosmetic. Dividing by sessions that reached the page stops late-funnel issues from being underrated. The result maps onto Nielsen's 0–4 severity scale.

Confidence	Rule
High	A deterministic signal plus 2+ personas, repeated across runs
Medium	LLM-identified, but repeated across sessions or personas
Low	One session only, or the same persona passed on other repetitions; flagged "needs human review"

Report generator

The report reads top-down from verdict to proof: how each task went, what broke, and the evidence for every issue one click away.

1. **Summary.** Success rate per task, the top five issues, and the study's cost and coverage.
2. **Task × persona matrix.** A heatmap of success rate, median steps and estimated human time per cell. It shows patterns like "mobile personas fail checkout" at a glance.
3. **Drop-off per task.** A funnel of how far each agent got, so the leak is visible.
4. **Issue cards**, sorted by severity.
5. **Accessibility appendix.** axe-core violations by page, plus what the screen-reader and keyboard personas ran into.
6. **Method and limits.** Personas, models, repetitions, and what synthetic users cannot tell you.

What an issue card holds

- Title, severity, confidence, and reach ("5 of 6 mobile sessions, 3 personas").
- The page and element, boxed on a before/after screenshot strip.
- One or two think-aloud quotes, each linked to its step.

- The signals that fired and the heuristic violated.
- "Watch it happen": the rrweb replay opened at that step's timestamp.
- A suggested fix, labelled as a suggestion.

Exports and integrations

- Shareable HTML report, and a PDF printed from the same HTML.
- JSON for CI, so a build can fail when a new high-severity issue appears.
- One-click Jira, Linear or GitHub issues with screenshots and the replay link attached.
- Run diff: compare two studies on the same tasks and label each issue new, fixed, regressed or unchanged. This is what turns the tool into part of a release loop.

Data model and storage

Nine entities cover the system, and every claim an issue makes resolves to Step rows and their stored artifacts.

Entity	Key fields	Stored in
Study	id, target_url, allowed_domains[], persona_mix, budget, status	Postgres
Task	id, study_id, goal_text, start_url, success_criteria (json)	Postgres
Persona	id, card (json), perception_profile, action_profile, budget	Postgres
Session	id, study_id, task_id, persona_id, seed, status, outcome, step_count, cost, started_at, ended_at	Postgres
Step	id, session_id, idx, url, state_hash, action (json), think_aloud, expectation, confidence, frustration, artifact keys, tokens	Postgres + object storage
Signal	id, step_id, type, details (json)	Postgres
Candidate issue	id, session_id, title, description, page, element, heuristic, step_ids[], embedding	Postgres + pgvector

Entity	Key fields	Stored in
Issue	id, study_id, title, severity, confidence, reach, persona_ids[], candidate_ids[], evidence_step_ids[], suggested_fix, status	Postgres
Report	id, study_id, version, compared_to_study_id, html_key, json_key	Postgres + object storage

A Session is one (Study, Task, Persona, seed); it owns many Steps, each Step owns its Signals, and each Issue groups the Candidate issues it was clustered from.

Tech stack

Playwright plus your own agent loop is the core; everything else is standard web infrastructure. For a hackathon, swap Temporal for BullMQ, Kubernetes for docker-compose, and start the agent from browser-use or Stagehand.

Layer	Recommended	Alternatives	Why
Browser automation	Playwright (Chromium; WebKit for Safari-like checks)	Puppeteer, Selenium	Isolated contexts, device emulation, network throttling, trace viewer
Agent loop	Your own loop on Playwright	browser-use (Python), Stagehand (TypeScript)	Perception filters and per-step evidence need full control; libraries are fine for a first prototype
Step model	Claude Haiku 5.5	Other fast vision models	Cheap, fast, reads screenshots
Critic and synthesis model	Claude Sonnet 5.5 or Opus 5.5	Other frontier models	Better judgement on long traces and merges
Pixel-only mode (optional)	Claude computer use via the API	—	Most human-like perception, for a few personas

Layer	Recommended	Alternatives	Why
Workflow engine	Temporal	BullMQ, Inngest, Celery	Durable, resumable sessions with retries
Browser fleet	Docker on Kubernetes	Browserbase, Steel, Browserless	Self-host for control; hosted for fast setup
Replay	rrweb + rrweb-player	Playwright video	Lightweight, seekable, DOM-faithful
Accessibility	axe-core	Lighthouse	Industry-standard rule set
Database	Postgres + pgvector	—	Relational data and embeddings in one place
Object storage	S3, Cloudflare R2 or MinIO	—	Cheap storage for screenshots and traces
Event bus	Redis Streams	NATS, Kafka	Feeds the live dashboard over WebSockets
Clustering	Embeddings + HDBSCAN	Agglomerative	No cluster count to pick up front
Frontend	Next.js, React, Tailwind	SvelteKit	Live dashboard and report pages
Observability	OpenTelemetry + Langfuse	Helicone	Infra traces and LLM calls side by side

Trust, cost and safety

The swarm is only useful if people believe its findings, so calibration is part of the product, not an afterthought.

Calibration

- Build a seeded-defect benchmark: a demo app with 10–20 known usability bugs (hidden CTA on mobile, misleading label, silent validation failure, unlabelled icon button). Track recall and precision on every model or prompt change.
- Where a team has past real usability studies, rerun the swarm on that build and compare issue lists.
- Route low-confidence issues to a human review queue; reviewer verdicts become examples that sharpen the critic.

Known agent biases and their counters

Bias	Effect	Counter
Over-competence	Under-reports issues real users hit	Skim filters, patience caps, jargon blind spots
Over-patience	Persists where people would quit	Hard retry cap per obstacle
Literalism	Flags wording nobody would trip on	High confidence needs a signal or repeats across personas
Hallucinated UI	Describes elements that aren't there	Cited steps validated against the element list and screenshot
Homogeneity	One model makes every persona think alike	Perception and action filters; mix models across personas

Cost

```
\text{cost} \approx N_{\text{sessions}} \times \bar{s} \times
\left(t_{\text{in}} \cdot p_{\text{in}} + t_{\text{out}} \cdot
p_{\text{out}}\right) + \text{critic and synthesis}
```

Here $\bar{s}$ is average steps per session, t is tokens per step and p is the model's current per-token price; screenshots dominate input tokens. The biggest levers:

- Prompt caching for the persona card and instructions, which repeat on every step.
- Downscaled screenshots, and none at all when the page hasn't changed.
- The cheap model for routine steps, escalating only when stuck.
- Early stop once a task is clearly blocked for that persona.

Safety rules

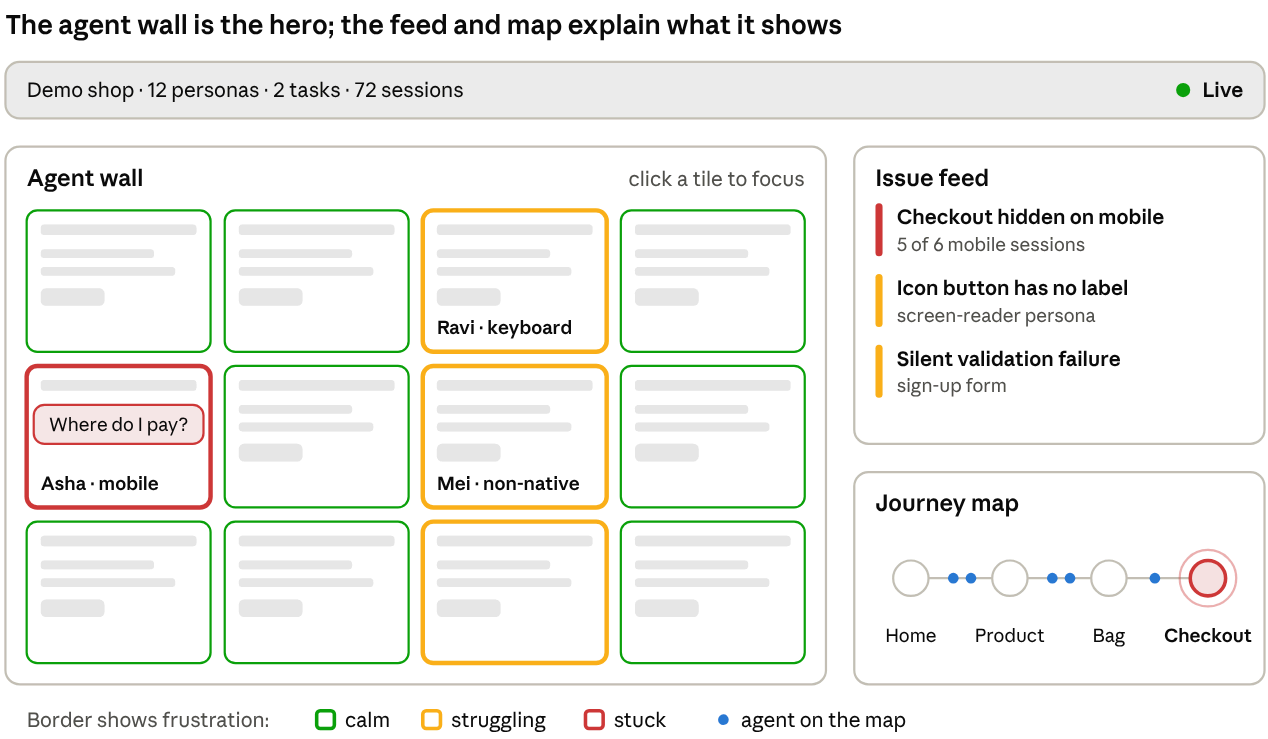
- Only test sites you own or have written permission to test. The proxy enforces the domain allowlist, so the agent cannot wander off it.
- No real money or real messages: payments run in sandbox or test mode, and actions like "Place order" on production are blocked unless the study allows them.
- Page content is data, never instructions. The action space has no way to send data elsewhere, which blunts prompt injection hidden in pages.
- Credentials and personal data stay out of model context and out of stored evidence.
- The swarm identifies itself honestly in its user agent and honours rate limits.

The visual demo

The demo is a live dashboard: the audience watches a dozen agents struggle in real time, sees issues form as agents converge on them, then watches a fix make them disappear.

Dashboard panels

- **Agent wall.** A grid of 12 live browser tiles. Each shows the persona's name, the current step, a think-aloud bubble, and a border that shifts green → amber → red with frustration.
- **Journey map.** Pages as nodes, agents as dots moving along the paths they take. Nodes pulse red where signals fire, so the bottleneck shows itself.
- **Issue feed.** Issues appear and merge live as clustering runs, each with a counter such as "hit by 5 of 6 mobile agents".
- **Focus view.** Click a tile to see that agent's marked screenshot, full think-aloud log and signal timeline.



live dashboard wireframe · 3 panels

Tile borders carry the frustration score, so the struggling agents catch the audience's eye before anyone explains them.

A three-minute demo script

Time	On screen	The point it makes
0:00–0:20	Paste the demo shop's URL, pick 12 personas and 2 tasks, press Launch	Setup takes seconds
0:20–1:20	The agent wall lights up, think-aloud streams, one tile turns red	Agents behave differently by persona
1:20–1:50	Focus on the mobile persona: the checkout button sits under the cookie banner	You see exactly why it failed
1:50–2:20	The issue feed merges "Checkout hidden on mobile", reach 5 of 6	Many agents, one root cause
2:20–2:40	Open the issue card, then the replay at the exact step	Every claim has evidence

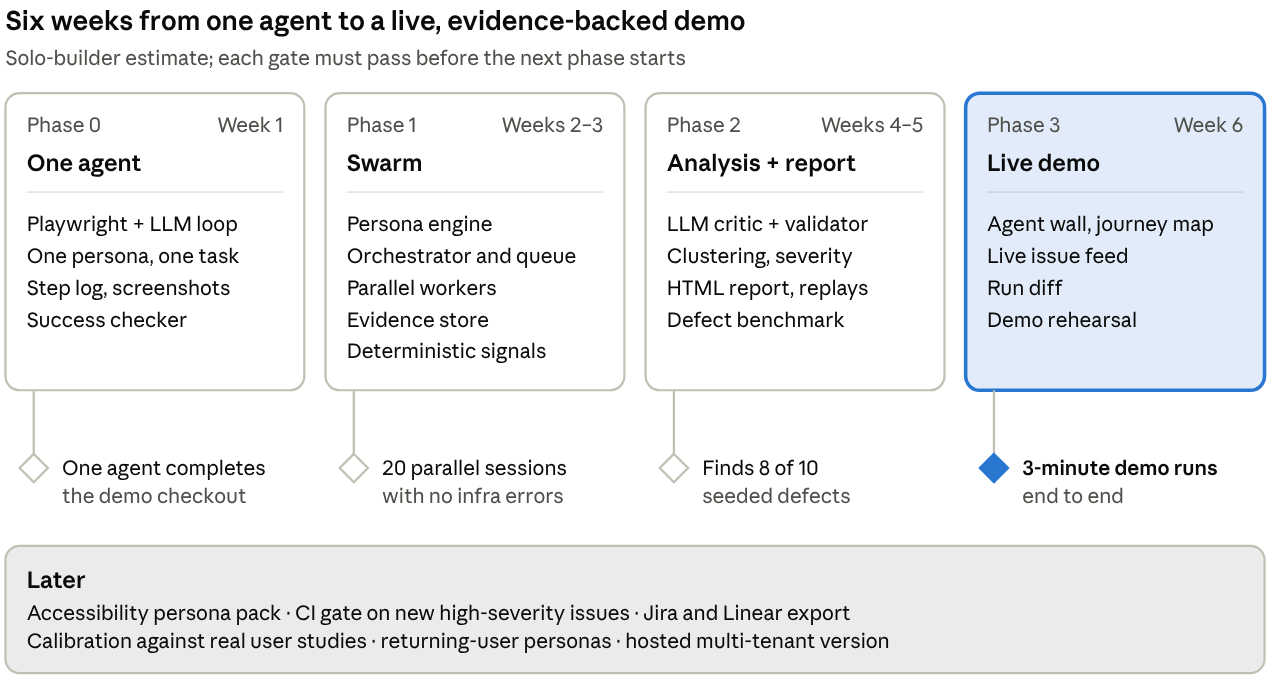
Time	On screen	The point it makes
2:40–3:00	Deploy the fix, rerun, the run diff marks the issue fixed	It closes the loop

Making it land

- Demo on your own site with seeded defects, so the run is predictable. Keep a recorded backup of a good run and say so if you use it.
- Pre-warm the browser pool so tiles light up the moment you press Launch.
- The before/after rerun is the moment people remember; protect the time for it.

Build roadmap

Build the single agent first, prove it can detect seeded defects, and only then polish the live demo.



build roadmap · 4 phases, 4 gates

Only have a weekend? Build Phase 0 on browser-use or Stagehand, run six personas in parallel with docker-compose, and show a simple tile wall plus a hand-reviewed issue list.

Risks and open questions

The biggest risk is trust: a few confident false alarms and teams stop reading the report.

Risk	Impact	Mitigation
Synthetic findings diverge from real users	Teams lose trust after false alarms	Seeded benchmark, confidence labels, positioned as a pre-filter for real research
Bot detection and CAPTCHAs	Sessions fail on protected sites	Test staging builds; have the site owner allowlist the swarm
Cost grows with swarm size	Large studies get expensive	Caching, screenshot diffing, model routing, cost caps
Non-determinism	Two runs disagree	Repetitions, fixed seeds and site state, reach instead of single hits
Site changes mid-study	Evidence mixes two builds	Pin a build or deploy id per study
Terms of service	Running on sites you don't own	Permission and a domain allowlist required to launch

- [ ] Should some personas remember past visits, to test returning-user flows?
- [ ] How much weight should frustration scores carry, given they are model self-reports?
- [ ] Which persona mix best predicts real-user findings? Only calibration data can answer this.
- [ ] If it becomes a product, is pricing per session, per study or per seat?

Sources

- UXAgent: An LLM Agent-Based Usability Testing Framework for Web Design, Lu et al., CHI 2025 late-breaking work
- browser-use, open-source Python browser-agent library
- Stagehand, open-source TypeScript browser-automation framework by Browserbase